UNIVERSIDAD CATÓLICA DE TEMUCO | FACULTAD DE INGENIERÍA

Ingeniería Civil en Informática

Teoría de la Computación – INFO1148

Proyecto: diseño e implementación de un analizador sintáctico

AgroRule: lenguaje de dominio específico para la automatización de un invernadero

Informe técnico y código fuente

Estudiantes Nicolás Vergara Von dem Knesebeck (jefe de grupo)
Rodrigo Altamirano Y.

Profesor Marcos Antonio Levano Huamaccto

Curso INFO1148 – 2026-2

Fecha 26 de octubre de 2026

Repositorio del proyecto: https://github.com/objval/agrorule-info1148

Índice

1. Introducción y dominio seleccionado **2**

2. Especificación del lenguaje **2**
2.1. Alfabeto y categorías de tokens . . . 2
2.2. Forma de una regla . . . 2
2.3. Ejemplos de sentencias . . . 3

3. Gramática libre de contexto **3**
3.1. Símbolo inicial, terminales y no terminales . . . 3
3.2. Producciones . . . 4

4. Análisis de la gramática **5**
4.1. Recursión izquierda . . . 5
4.2. Factorización por la izquierda . . . 5
4.3. Ambigüedad . . . 5
4.4. Conjuntos FIRST y FOLLOW . . . 6
4.5. Verificación LL(1) . . . 6

5. Tabla LL(1) **7**

6. Diseño e implementación del parser **9**
6.1. Estructura del código . . . 9
6.2. Cómo se usa la tabla en el parser . . . 9
6.3. Complejidad y comportamiento . . . 9

7. Árbol sintáctico **10**

8. Manejo de errores **10**

9. Casos de prueba **11**

10. Extensión creativa **13**

11. Conclusiones **14**

12. Referencias **14**

A. Instrucciones de ejecución **15**

B. Repositorio y organización del código **15**

1. Introducción y dominio seleccionado

Un invernadero automatizado recibe lecturas de sensores (temperatura, humedad de suelo, humedad ambiental, luz, pH, lluvia) y actúa sobre mecanismos (riego, ventiladores, mallas, techo, calefacción). La lógica que une unas con otros suele escribirse como reglas del tipo "si pasa esto, haz aquello". En la práctica esas reglas terminan dentro del código de un controlador, mezcladas con la lectura de sensores, y cualquier cambio de criterio obliga a modificar y volver a desplegar el programa completo.

Este proyecto define un lenguaje de dominio específico para escribir esas reglas por separado y un analizador sintáctico que verifica que estén bien escritas antes de que el controlador las ejecute. El lenguaje se llama **AgroRule**. El alcance del proyecto es el análisis sintáctico, no la ejecución de las reglas, y por eso el resultado del analizador es un árbol sintáctico abstracto y no una secuencia de acciones ejecutadas.

El dominio se eligió por tres razones. Primero, las reglas de un invernadero se parecen mucho entre sí, lo que permite una gramática pequeña y un lenguaje que se aprende en minutos. Segundo, el vocabulario es concreto y verificable, con magnitudes físicas y actuadores reconocibles. Tercero, el lenguaje necesita condiciones compuestas, porque casi ninguna decisión agronómica depende de una sola variable. Esa necesidad obliga a resolver precedencia, asociatividad y anidamiento, que es justamente el contenido del curso.

2. Especificación del lenguaje

2.1. Alfabeto y categorías de tokens

El alfabeto del lenguaje incluye letras, dígitos, el guion bajo, los operadores relacionales, los paréntesis, la coma, el punto y coma, las comillas y el símbolo de comentario. El analizador léxico agrupa esos caracteres en las categorías de la Tabla 1.

Cada token conserva la línea y la columna donde empieza. Esa decisión no es cosmética: todos los mensajes de error del sistema apuntan a una posición concreta de la entrada, y sin esa información el usuario solo sabría que algo está mal, no dónde.

2.2. Forma de una regla

Una regla tiene cuatro partes. La palabra `WHEN` abre la regla, una condición indica cuándo se aplica, `THEN` separa la condición de las acciones y `END` cierra la regla. Dentro de la condición se comparan dos términos con un operador relacional, y esas comparaciones se pueden unir con `AND`, `OR`, negar con `NOT` y agrupar con paréntesis. Las acciones son identificadores que pueden llevar argumentos entre paréntesis y se separan con punto y coma. Un programa es una secuencia de reglas.

Cuadro 1: Categorías de tokens de AgroRule.

Categoría	Ejemplos	Descripción
Palabras reservadas	`WHEN`, `THEN`, `AND`, `OR`, `NOT`, `END`, `TRUE`, `FALSE`	Ocho palabras con significado fijo, escritas en mayúsculas
Identificador	`temperatura`, `abrir_ventilador`, `ph_suelo`	Letra o guion bajo seguido de letras, dígitos o guion bajo
Número	`28`, `6.5`, `1.5`	Entero o decimal con punto
Booleano	`TRUE`, `FALSE`	Valor de verdad, se comporta como palabra reservada
Texto	`"calor"`, `"nitrato"`	Cadena entre comillas dobles, sin salto de línea
Operador relacional	`>`, `<`, `>=`, `<=`, `==`, `!=`	Compara dos términos
Paréntesis	`(`, `)`	Agrupan condiciones
Separadores	`;`, `,`	Separan acciones y argumentos
Comentario	`# texto`	Se ignora hasta el final de la línea

2.3. Ejemplos de sentencias

Cuadro 2: Cinco sentencias válidas y cinco inválidas del lenguaje.

N	Sentencia	Comentario
V1	`WHEN temperatura > 28 THEN abrir_ventilador END`	Regla mínima
V2	`WHEN humedad < 30 AND temperatura > 25 THEN regar(20) END`	Condición compuesta con AND
V3	`WHEN NOT (temperatura > 30) THEN cerrar_techo END`	Negación de un grupo
V4	`WHEN ph >= 6.5 AND ph <= 7.5 THEN fertirriego(1.5, "nitrato") END`	Argumento numérico y de texto
V5	`WHEN suelo == seco OR lluvia == FALSE THEN regar(15); avisar("riego") END`	Dos acciones en una regla
I1	`WHEN temperatura > 28 abrir_ventilador END`	Falta THEN
I2	`WHEN temperatura > 28 THEN abrir_ventilador`	Falta END
I3	`WHEN temperatura > THEN regar(20) END`	Falta el valor de la derecha
I4	`WHEN temperatura > 28 THEN regar(20) abrir_ventilador END`	Falta el punto y coma entre acciones
I5	`WHEN temperatura 28 THEN regar(20) END`	Falta el operador relacional

3. Gramática libre de contexto

3.1. Símbolo inicial, terminales y no terminales

La gramática se define sobre el conjunto de tokens, no sobre caracteres, porque el analizador léxico ya resolvió esa parte. El símbolo inicial es `programa`. La gramática tiene 19 no terminales, 16 símbolos terminales y el centinela de fin de entrada ($). Los no terminales con apóstrofo (`disy'`, `conj'`, `acciones'`, `args'`) representan la repetición de una estructura y existen para evitar la recursión

izquierda.

3.2. Producciones

Cuadro 3: Producciones de la gramática de AgroRule.

ID	Producción
P1	programa → lista_reglas
P2	lista_reglas → regla lista_reglas
P3	lista_reglas → ε
P4	regla → WHEN cond THEN acciones END
P5	cond → disy
P6	disy → conj disy'
P7	disy' → OR conj disy'
P8	disy' → ε
P9	conj → neg conj'
P10	conj' → AND neg conj'
P11	conj' → ε
P12	neg → NOT neg
P13	neg → prim
P14	prim → LPAREN cond RPAREN
P15	prim → comp
P16	comp → term OP_REL term
P17	term → IDENT
P18	term → NUM
P19	term → TRUE
P20	term → FALSE
P21	acciones → accion acciones'
P22	acciones' → SEMI accion acciones'
P23	acciones' → ε
P24	accion → IDENT llamada
P25	llamada → LPAREN args RPAREN
P26	llamada → ε
P27	args → arg args'
P28	args' → COMMA arg args'
P29	args' → ε
P30	arg → NUM
P31	arg → TRUE
P32	arg → FALSE

ID	Producción
P33	arg → IDENT
P34	arg → CADENA

La gramática tiene 34 producciones. Trece de ellas corresponden a los no terminales que repiten estructuras (condiciones, acciones y argumentos) y el resto a la estructura fija de una regla. El orden de las producciones no influye en el análisis, porque el parser decide por token de adelanto y no por orden de escritura.

4. Análisis de la gramática

4.1. Recursión izquierda

La gramática no tiene recursión izquierda. El caso crítico son las condiciones compuestas: si se escribiera `disy` → `disy OR conj`, el analizador de descenso recursivo llamaría a `disy` con la misma entrada y entraría en un ciclo infinito. Para evitarlo, la repetición se expresa con un no terminal auxiliar a la derecha, como en `disy` → `conj disy'` y `disy'` → `OR conj disy'` | ε. Esa forma produce una asociatividad por la izquierda en el árbol resultante sin recursión por la izquierda en la gramática.

4.2. Factorización por la izquierda

Hay un solo lugar donde la factorización fue necesaria. Con las producciones `accion` → `IDENT args` y `accion` → `IDENT` el parser no podría decidir cuál aplicar mirando un identificador, porque ambas empiezan con el mismo símbolo. La solución fue sacar el prefijo común: `accion` → `IDENT llamada` y `llamada` → `( args )` | ε. Ahora la decisión se toma después de leer el identificador, cuando el siguiente token es un paréntesis o no lo es.

4.3. Ambigüedad

La gramática no es ambigua. La precedencia entre `AND` y `OR` no se deja al azar ni a una convención implícita: está en la estructura de la gramática. El no terminal `disy` agrupa conjunciones y el no terminal `conj` agrupa negaciones, de modo que `AND` se resuelve antes que `OR` y los paréntesis permiten alterar ese orden. Una gramática plana del tipo `cond` → `cond AND cond` | `cond OR cond` sería ambigua y además recursiva por la izquierda.

El caso de las acciones se resolvió de forma parecida. Cada acción termina en un punto y coma o en `END`, y esos dos símbolos no aparecen dentro de una acción, así que el límite entre acciones nunca es ambiguo.

4.4. Conjuntos FIRST y FOLLOW

Cuadro 4: Conjuntos FIRST y FOLLOW por no terminal.

No terminal	FIRST	FOLLOW
programa	WHEN, ε	$
lista_reglas	WHEN, ε	$
regla	WHEN	WHEN, $
cond	NOT, (, id, num, TRUE, FALSE	THEN,)
disy	NOT, (, id, num, TRUE, FALSE	THEN,)
disy'	OR, ε	THEN,)
conj	NOT, (, id, num, TRUE, FALSE	THEN, OR,)
conj'	AND, ε	THEN, OR,)
neg	NOT, (, id, num, TRUE, FALSE	THEN, OR, AND,)
prim	(, id, num, TRUE, FALSE	THEN, OR, AND,)
comp	id, num, TRUE, FALSE	THEN, OR, AND,)
term	id, num, TRUE, FALSE	THEN, OR, AND,), op
acciones	id	END
acciones'	;, ε	END
accion	id	END, ;
llamada	(, ε	END, ;
args	id, num, TRUE, FALSE, txt	)
args'	„ ε	)
arg	id, num, TRUE, FALSE, txt	), ,

Dos conjuntos explican por qué la gramática es LL(1). FIRST de `cond` es {`NOT`, `(`, `id`, `num`, `TRUE`, `FALSE`} porque una condición puede empezar con una negación, con un paréntesis o directamente con una comparación. FOLLOW de `disy'` es {`THEN`, `)`}, porque un `disy'` termina cuando aparece `THEN` o el paréntesis que cierra la condición que lo contiene. Esos dos conjuntos no se cruzan con FIRST de la otra alternativa de `disy'`, que es {`OR`}, y por eso la decisión es única.

4.5. Verificación LL(1)

La tabla predictiva se construyó con un algoritmo que, para cada producción, calcula su conjunto de predicción y detecta si dos producciones del mismo no terminal compiten por el mismo terminal. El resultado fue cero conflictos. La comprobación no es una revisión manual: el mismo cálculo que genera la tabla del informe es el que usa el analizador en tiempo de ejecución, y una prueba automática falla si aparece un conflicto nuevo.

5. Tabla LL(1)

La Tabla 5 se presenta en orientación horizontal porque tiene diecisiete columnas. Cada celda indica la producción que el analizador aplica cuando el no terminal de la fila aparece con el token de la columna en la entrada. Las celdas vacías representan combinaciones que la gramática no admite, y son las que generan los mensajes de error.

La lectura de la tabla es directa. En la fila `programa` la única celda ocupada es la del centinela $, porque un programa empieza a analizarse cuando llega el primer token y la producción P1 se aplica sin consumir nada. En la fila `regla` todas las celdas están vacías salvo `WHEN`, lo que confirma que una regla no puede empezar de otra forma. En la fila `neg` aparecen dos celdas, P12 con `NOT` y P13 con el resto de los símbolos que pueden iniciar una condición, y esa separación es la que permite que `NOT NOT` se lea sin ambigüedad.

Hay dos filas que conviene mirar juntas. `disy'` solo tiene P7 en la columna `OR` y P8 en `THEN` y `)`, mientras `conj'` tiene P10 en `AND` y P11 en `THEN`, `OR` y `)`. La diferencia entre ambas filas es exactamente la precedencia: `OR` cierra una conjunción pero no al revés, y por eso `a > 1 OR b > 2 AND c > 3` se agrupa como se espera.

La tabla también explica el comportamiento del analizador frente a un error. Cuando la celda correspondiente está vacía, el parser no elige una producción al azar ni intenta retroceder: informa los símbolos que la fila sí admitía. Un `then` que aparece antes de cerrar la condición produce un mensaje con `AND`, `OR` y el paréntesis de cierre como alternativas, porque esas son las columnas ocupadas de la fila `conj'`.

Cuadro 5: Tabla predictiva LL(1). Cada celda indica la producción que se aplica.

No terminal	WHEN	THEN	END	OR	AND	NOT	(	)	op	id	num	TRUE	FALSE	;	,	txt	$
programa	P1																P1
lista_reglas	P2																P3
regla	P4																
cond						P5	P5			P5	P5	P5	P5				
disy						P6	P6			P6	P6	P6	P6				
disy'		P8		P7				P8									
conj						P9	P9			P9	P9	P9	P9				
conj'		P11		P11	P10			P11									
neg						P12	P13			P13	P13	P13	P13				
prim							P14			P15	P15	P15	P15				
comp										P16	P16	P16	P16				
term										P17	P18	P19	P20				
acciones										P21							
acciones'			P23											P22			
accion										P24							
llamada			P26				P25							P26			
args										P27	P27	P27	P27			P27	
args'								P29							P28		
arg										P33	P30	P31	P32			P34	

6. Diseño e implementación del parser

6.1. Estructura del código

El código está separado en módulos con una responsabilidad cada uno. La Tabla 6 resume esa separación.

Cuadro 6: Módulos del analizador y su responsabilidad.

Módulo	Responsabilidad
`lexer.py`	Convierte el texto en tokens con línea y columna. Reconoce comentarios, números, textos e identificadores
`grammar.py`	Declara las producciones una sola vez y calcula FIRST, FOLLOW, la tabla LL(1) y los conflictos
`parser.py`	Analizador por descenso recursivo. Una función por no terminal, decisión por tabla
`ast_nodes.py`	Nodos del árbol, dibujo en texto y exportación a Graphviz
`errors.py`	Errores léxicos y sintácticos con posición, símbolo esperado y símbolo encontrado
`cli.py`	Interfaz de línea de comandos con opciones para árbol, tokens, tabla y exportación

No se usó ningún generador de analizadores. La gramática se declara como una lista de tuplas y los algoritmos de FIRST, FOLLOW y tabla predictiva están escritos en el mismo módulo, con unas cien líneas en total.

6.2. Cómo se usa la tabla en el parser

Cada función del parser corresponde a un no terminal. Cuando ese no terminal tiene más de una producción, la función no decide con una cadena de condicionales sobre el texto: consulta `TABLA[no_terminal][token.tipo]` y aplica la producción que la tabla indica. Si la celda está vacía, el error se construye con la lista de tokens que la tabla sí admitía en esa fila.

Esa decisión tiene una consecuencia práctica. El mensaje de error y la gramática no pueden desincronizarse, porque ambos salen de la misma estructura. Si mañana se agrega una producción, la tabla se recalcula y los mensajes de error incluyen el símbolo nuevo sin tocar el manejador de errores.

6.3. Complejidad y comportamiento

El analizador recorre la entrada una sola vez, con un token de adelanto y sin retroceder. El costo es lineal en la cantidad de tokens. La profundidad de recursión crece con el anidamiento de paréntesis y negaciones, y una prueba automática verifica que cuarenta paréntesis anidados se procesan sin problemas.

7. Árbol sintáctico

El resultado del análisis es un árbol sintáctico abstracto. La diferencia con un árbol de derivación es deliberada: el árbol de derivación conserva cada paso de la gramática, incluidos `WHEN`, `THEN` y `END`, mientras que el AST conserva solo la estructura semántica. Un `AND` queda como un nodo con dos hijos, una comparación como un nodo con operador y dos términos, y una acción como un nodo con nombre y argumentos.

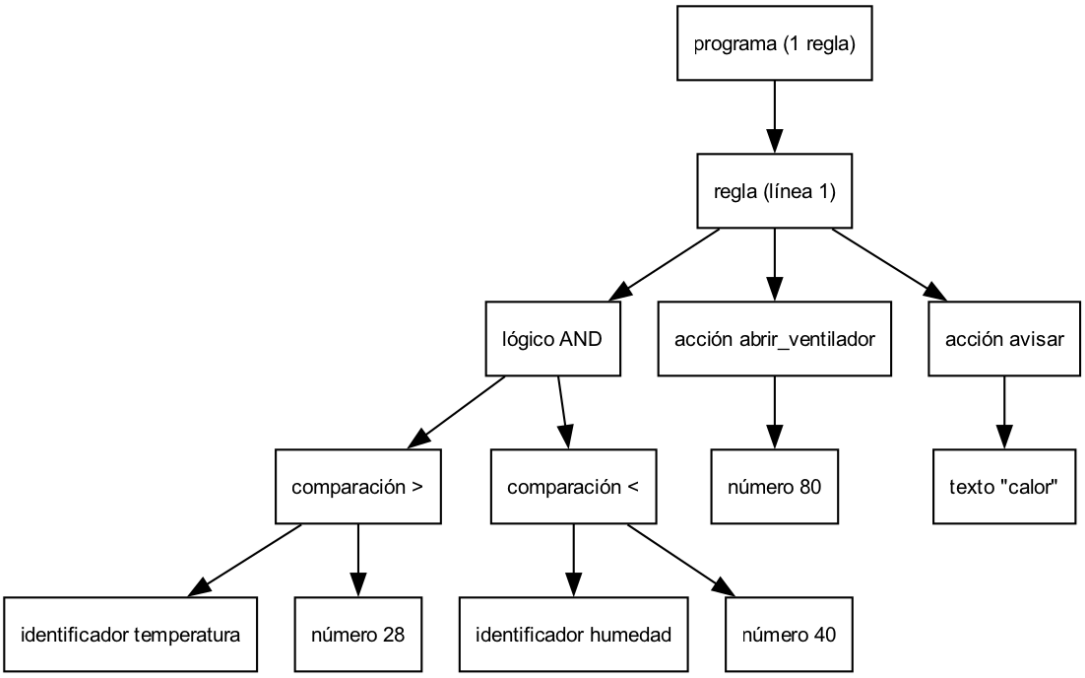


Figura 1: AST generado para `WHEN temperatura > 28 AND humedad < 40 THEN abrir_ventilador(80); avisar("calor") END`.

El árbol se puede imprimir en texto con líneas guía o exportar a Graphviz. La Figura 1 es la salida de Graphviz para una regla con dos acciones, y permite ver tres cosas: la condición se agrupó como `AND` de dos comparaciones, cada comparación tiene su operador como etiqueta y las acciones cuelgan de la regla en el orden en que se escribieron.

8. Manejo de errores

El analizador distingue tres familias de error. La primera es léxica y aparece cuando hay un carácter que no pertenece al alfabeto o un texto sin cerrar. La segunda es sintáctica y aparece cuando la secuencia de tokens no corresponde a ninguna producción aplicable. La tercera es de fin de entrada, cuando el programa termina antes de cerrar una regla o una lista de argumentos.

Todos los mensajes incluyen línea, columna y una explicación en palabras del dominio. Algunos ejemplos reales de salida son los siguientes.

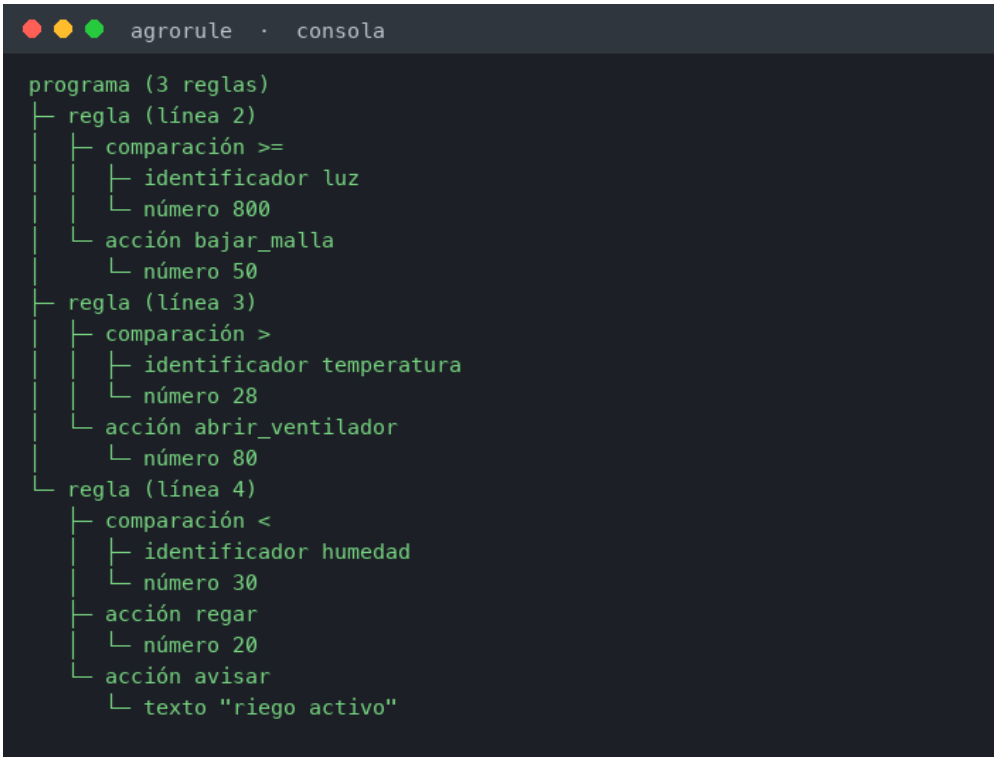


Figura 2: Salida del analizador para un archivo con tres reglas.

```
Error sintáctico (línea 1, columna 23): se esperaba AND, OR o el final de la
condición (THEN o un paréntesis de cierre). Se encontró: identificador.

Error sintáctico (línea 1, columna 20): se esperaba un valor (número,
identificador, TRUE o FALSE). Se encontró: THEN.

Error sintáctico (línea 1, columna 38): se esperaba ';' entre acciones o END
para cerrar la regla. Se encontró: identificador.

Error léxico (línea 1, columna 44): carácter no permitido: '@'.
```

El mensaje no se limita a decir que hay un error. Indica qué construcción se estaba leyendo, qué símbolos habrían permitido continuar y qué símbolo apareció. Ese conjunto de símbolos esperados es exactamente la fila de la tabla LL(1) para ese no terminal.



Figura 3: Mensaje de error para una regla sin THEN.

9. Casos de prueba

La Tabla 7 se generó ejecutando el analizador sobre cada entrada, no escribiendo los resultados a mano. Las diez primeras son válidas, las diez siguientes son inválidas y las cinco últimas son casos

límite.

Cuadro 7: Casos de prueba ejecutados sobre el analizador.

N	Entrada	Resultado obtenido
Válidas		
1	`WHEN temperatura > 28 THEN abrir_ventilador END`	válida, 1 regla
2	`WHEN humedad < 30 AND temperatura > 25 THEN regar(20) END`	válida, 1 regla
3	`WHEN suelo == seco OR lluvia == FALSE THEN regar(15); avisar("riego activo") END`	válida, 1 regla
4	`WHEN NOT (temperatura > 30) THEN cerrar_techo END`	válida, 1 regla
5	`WHEN luz >= 800 THEN bajar_malla(50); encender_extractor END`	válida, 1 regla
6	`WHEN temperatura > 28 THEN abrir_ventilador END WHEN humedad < 30 THEN regar(20) END`	válida, 2 reglas
7	`WHEN CO2 > 900 AND humedad > 80 THEN ventilar(TRUE) END`	válida, 1 regla
8	`WHEN NOT NOT lluvia == TRUE THEN cerrar_techo END`	válida, 1 regla
9	`WHEN ((temperatura > 28)) THEN abrir_ventilador(75) END`	válida, 1 regla
10	`WHEN ph >= 6.5 AND ph <= 7.5 THEN fertirriego(1.5, "nitrato") END`	válida, 1 regla
Inválidas		
11	`WHEN temperatura > 28 abrir_ventilador END`	inválida: se esperaba AND, OR o el final de la condición (THEN o un paréntesis de cierre). Se encontró: identificador.
12	`WHEN temperatura > 28 THEN abrir_ventilador`	inválida: se esperaba '(' con los argumentos, ';' para otra acción o END para cerrar la regla. Se encontró: fin de entrada.
13	`WHEN temperatura > THEN abrir_ventilador END`	inválida: se esperaba un valor (número, identificador, TRUE o FALSE). Se encontró: THEN.
14	`WHEN temperatura > 28 THEN END`	inválida: se esperaba el nombre de una acción. Se encontró: END.
15	`WHEN temperatura > 28 THEN regar(20) abrir_ventilador END`	inválida: se esperaba ';' entre acciones o END para cerrar la regla. Se encontró: identificador.
16	`WHEN THEN regar(20) END`	inválida: se esperaba una condición formada por dos valores y un operador relacional. Se encontró: THEN.

N	Entrada	Resultado obtenido
17	`WHEN (temperatura > 28 THEN regar(20) END`	inválida: se esperaba un paréntesis de cierre. Se encontró: THEN.
18	`WHEN temperatura > 28 THEN regar(20) END @`	inválida: carácter no permitido: '@'
19	`WHEN temperatura > 28 THEN regar(20, ) END`	inválida: se esperaba un argumento (número, TRUE, FALSE, identificador o texto). Se encontró:).
20	`WHEN temperatura 28 THEN regar(20) END`	inválida: se esperaba un operador relacional (>, <, >=, <=, ==, !=). Se encontró: número.
Casos límite		
21	`(vacío)`	válida, 0 reglas
22	`# solo comentarios`	válida, 0 reglas
23	`WHEN NOT NOT NOT temperatura > 5 THEN calentar END`	válida, 1 regla
24	`WHEN temperatura > 5 THEN avisar END`	válida, 1 regla
25	`WHEN temperatura > 5 THEN regar(1); regar(2); regar(3); cerrar_techo END`	válida, 1 regla

Los cinco casos límite se eligieron por lo que revelan y no por lo que confirman. El programa vacío prueba que la lista de reglas admite épsilon sin consumir tokens. El archivo con solo comentarios prueba que el analizador léxico descarta el contenido sin dejar tokens huérfanos. Las negaciones encadenadas prueban que la producción `neg` → `NOT neg` no entra en conflicto con `neg` → `prim`. La acción sin paréntesis prueba la producción épsilon de `llamada`. Las cuatro acciones seguidas prueban la recursión de `acciones'`. Además, la batería de pruebas incluye un anidamiento de cuarenta paréntesis y un texto de quinientos caracteres para verificar que el analizador no depende del tamaño de la entrada.

10. Extensión creativa

El lenguaje incorpora cinco extensiones sobre el mínimo exigido. Cada una se refleja en la gramática, en los conjuntos FIRST y FOLLOW, en la tabla LL(1), en el código y en las pruebas.

1. **Negación con `NOT`.** Agrega la producción `neg` → `NOT neg` y permite frases como `WHEN NOT (temperatura > 30) THEN cerrar_techo END`. Introduce la única recursión a la derecha de la gramática.

2. **Paréntesis.** Agrega `prim` → `( cond )` y permite alterar la precedencia entre `AND` y `OR`. El paréntesis de cierre aparece en FOLLOW de `cond` y de `disy'`.

3. **Acciones con argumentos tipados.** Agrega `args` y `arg` con cinco tipos posibles de argumento (número, booleano, identificador y texto). Una acción puede llevar cero, uno o varios argumentos.

4. **Múltiples acciones por regla.** Agrega `acciones → accion acciones'` con separador punto y coma, lo que permite escribir secuencias como `regar(20); avisar("riego activo")`.

5. **Comentarios.** El analizador léxico descarta desde `#` hasta el final de la línea. No afecta a la gramática, pero sí al análisis léxico y a los números de línea que usan los errores.

La extensión más costosa fue la segunda, porque obligó a decidir si el paréntesis pertenece a la condición o a la comparación. Quedó en la condición, con lo que una comparación nunca contiene paréntesis internos y la gramática se mantiene LL(1).

11. Conclusiones

El proyecto entrega un lenguaje pequeño con una gramática verificable y un analizador que se puede justificar línea por línea. La gramática tiene 34 producciones, 19 no terminales y ningún conflicto LL(1), y esa condición no se afirma: la calcula el mismo código que ejecuta el análisis.

La decisión de diseño que más simplificó el resto fue sacar la variabilidad de la gramática hacia no terminales auxiliares. Al escribir las repeticiones con apóstrofos, la recursión izquierda desapareció, la tabla predictiva quedó completa y el parser pudo ser un descenso recursivo de una función por no terminal. La factorización de la acción cumplió el mismo papel en el único punto donde dos producciones compartían prefijo.

Quedan dos limitaciones explícitas. El lenguaje no tiene expresiones aritméticas, así que no se puede escribir `temperatura > maxima + 2`. Y el analizador verifica sintaxis, no tipos: acepta `regar("veinte")` porque el texto es un argumento válido, aunque el actuador espere un número. Ambas cosas se pueden agregar sin tocar la estructura del parser, con una gramática de expresiones y una tabla de tipos respectivamente.

12. Referencias

1. Aho, A., Lam, M., Sethi, R. y Ullman, J. *Compiladores: principios, técnicas y herramientas*. Pearson, segunda edición.

2. Hopcroft, J., Motwani, R. y Ullman, J. *Introducción a la teoría de autómatas, lenguajes y computación*. Pearson.

3. INFO1148. *Proyecto: diseño e implementación de un analizador sintáctico*. Educa Blackboard, 2026.

4. Documentación de Python 3. https://docs.python.org/3/

A. Instrucciones de ejecución

El código se ejecuta con Python 3.10 o superior y no requiere bibliotecas externas para el análisis. Los comandos principales son los siguientes.

Cuadro 8: Comandos de la interfaz de línea de comandos.

Comando	Propósito
`python3 -m agrorule archivo.ar`	Analizar un archivo y confirmar validez
`python3 -m agrorule --texto "..." --ast`	Analizar texto directo y mostrar el árbol
`python3 -m agrorule archivo.ar --tokens`	Mostrar los tokens con posición
`python3 -m agrorule archivo.ar --dot ast.dot`	Exportar el AST a Graphviz
`python3 -m agrorule --tabla`	Imprimir FIRST, FOLLOW y la tabla LL(1)
`python3 -m unittest discover -s tests -v`	Ejecutar las trece pruebas

B. Repositorio y organización del código

El código fuente, las pruebas, los ejemplos y los scripts que generan las tablas de este informe están en el repositorio del proyecto: https://github.com/objval/agrorule-info1148.

La estructura de carpetas es la siguiente.

Cuadro 9: Contenido del repositorio.

Ruta	Contenido
`agrorule/`	Paquete con el analizador léxico, la gramática, el parser, el AST y la interfaz
`tests/`	Trece pruebas automáticas
`examples/validos/`	Cuatro programas válidos de ejemplo
`examples/invalidos/`	Cinco programas con errores típicos
`scripts/`	Generación de tablas, imagen del árbol y capturas
`README.md`	Instrucciones, dependencias, formato de entrada y resultados

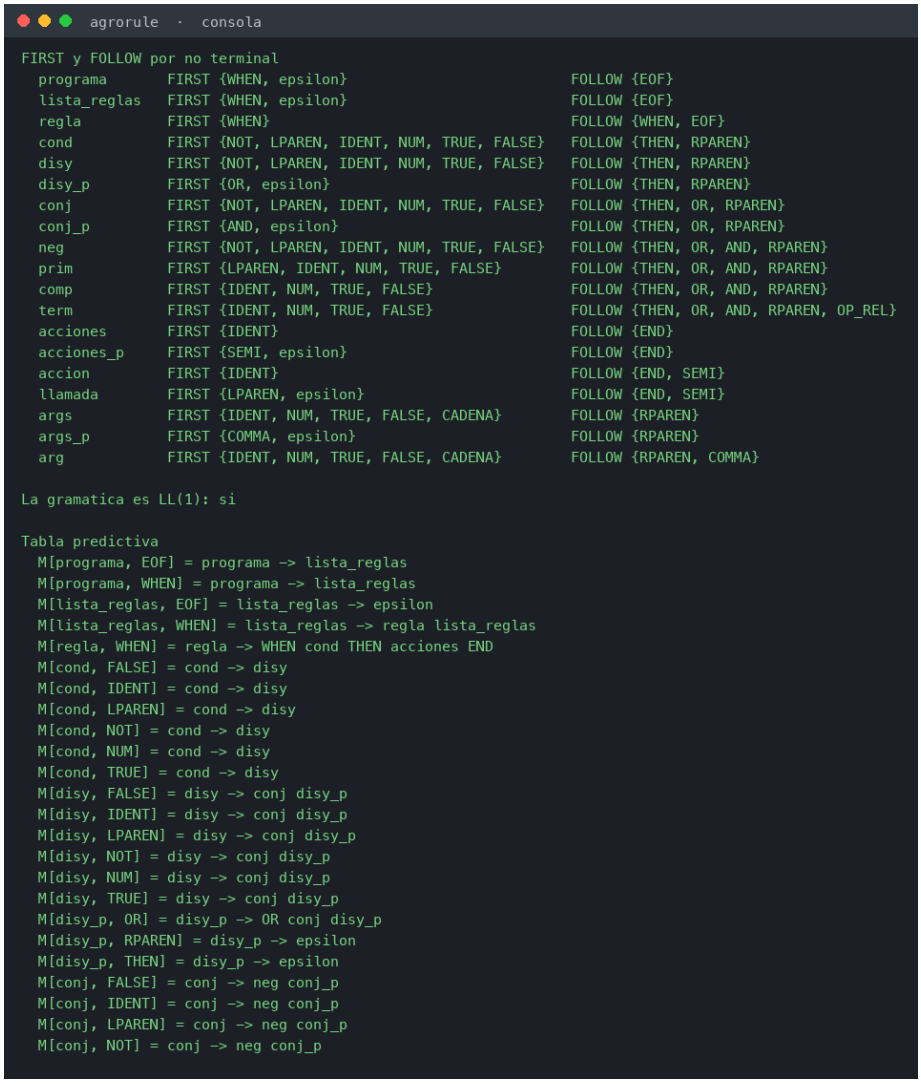


Figura 4: Salida de `python3 -m agrorule --tabla`: FIRST, FOLLOW, verificación LL(1) y primeras filas de la tabla predictiva.